

SONiC物理スイッチ上での MPLS実験環境 構築・デバッグ総合レポート

ー パケット優先制御・接続障害の原因究明・自動リルーティング設計 ー

2026年6月 frr-docker-lab プロジェクト

構成: 1.やりたかったこと 2.技術のやさしい解説 3.発生した問題 4.原因究明(3段階) 5.解決策 6.試行錯誤の記録 7.最終結果 8.技術的まとめ 9.学んだこと 10.QoS実証結果 11.Fast Reroute実証結果

1 そもそも何をやりたかったのか

このプロジェクトには3つの大きな目標がありました。

☑ 目標① 実験ネットワークの構築

2台の物理スイッチ(Edgecore AS7326-56X)の上にDockerコンテナを起動し、仮想的なネットワーク機器(ルータ・端末)として動かす。「本物のハードウェア上で動くソフトウェアルータ」という実験環境を作る。

☑ 目標② パケットの優先度制御(QoS)

ネットワークが混雑しているとき、重要なデータを優先して通す仕組みを作る。救急車が一般車を追い越して先に進めるように、優先度が高いパケットほど「スループット大・遅延小・パケロス小」になるようにしたい。

☑☑ 目標③ 障害時の自動迂回(Fast Reroute)

途中の回線が故障したとき、カーナビが道路工事を検知して別ルートを案内するように、ネットワークも自動で障害を検知し、正常な経路に切り替えたい。人間が手動で設定し直す前に、数十ミリ秒以内で自動復旧することが目標。

■ 実験で使うネットワーク構成

2台のスイッチにそれぞれコンテナを配置し、MPLSで転送します。

スイッチ	コンテナ名	役割
SW1	Tx1 / Tx2 / Tx3	送信端末(トラフィックを発生させる)
SW1	LER_Ingress	MPLS入口ルータ:ラベルをPUSH(貼り付け)する
SW1	CR1 / CR2 / CR3	コアルータ:ラベルをSWAP(書き換え)して中継する
SW2	LER_Egress	MPLS出口ルータ:ラベルをPOP(剥がす)する
SW2	Rx1 / Rx2 / Rx3	受信端末(iperf3サーバ、帯域を測定する)

パケットの流れ:Tx1 → LER_Ingress → CR1 → LER_Egress → Rx1 (Tx2/Tx3も同様)

※ TTL=61(64から3減)→ 3つのルータ(ホップ)を経由した証拠

2 使った技術のやさしい解説

専門用語を日常の例え話で説明します。

☒ MPLS (エムピーエルエス)

宅配便の「送り状ラベル」のようなもの。パケットにラベルを貼り付け、中継ルータ (CR1/CR2/CR3) はIPアドレスを見ずにラベルの番号だけで転送先を決める。仕分けセンターのスクナが伝票番号だけで行き先を決めるイメージ。処理が単純なので高速・効率的。

☒ SONiC (ソニック)

スイッチのOS (オペレーティングシステム)。WindowsやLinuxと同じようにスイッチのハードウェアを制御するソフトウェア。Microsoftが開発してオープンソース化。Googleなど大手クラウド企業のデータセンターでも使われている。

☒ ASIC (エーシック)

ネットワーク専用の超高速処理チップ (Broadcom Trident3を使用)。普通のCPUが「何でもできる汎用コンピュータ」だとすれば、ASICは「パケット転送だけを猛烈に速くやる専用マシン」。

☒ KNET (ケーネット)

ASICとLinuxカーネルをつなぐ「窓口」。外から届いたパケットをASICが受け取り、KNET経由でLinux上のDockerコンテナに渡す。宅配センターの「仕分け係」のような存在。

☒☒ ebtables (イービーテーブルズ)

Linuxのブリッジ (L2スイッチ相当) に対するファイアウォール。「このような種類のパケットは通すな」というルールを設定できる。SONiCはセキュリティ上の理由でデフォルトで多くのルールが入っており、今回の問題の原因の一つになった。

☒ tc ingress (ティーシー イングレス)

Linuxカーネルが持つ「受信パケットへの最初の処理」機能。パケットが届いた瞬間、他のどんな処理より前に実行される。今回はここでVLANタグを剥がすことで問題を解決した (詳しくはSection 5)。

3 問題になっていたこと

スクリプトで環境を構築してもコンテナ間で通信が全くできませんでした。

症状	pingを実行すると「Destination Host Unreachable」と表示され100%パケロス
確認方法	tcpdump (ネットワーク監視ツール) でパケットの流れを観察
発見したこと	ARPリクエスト (問い合わせ) は送信できているが、ARPリプライ (返事) が届かない
影響範囲	SW1内部・SW2内部・スイッチ間リンク、すべてのパスが不通

☒ ARPとは何か(なぜ重要か)

ARP (Address Resolution Protocol) は「この電話番号 (IPアドレス) の人の住所 (MACアドレス) を教えて」という問い合わせプロトコル。電話をかける前に住所録を調べるようなもの。ARPが通らないと、pingはもちろん、その先のすべての通信が不可能になる。

4 原因の究明(3段階)

問題は1つではなく、複数の原因が重なっていました。tcpdumpで段階的に切り分けました。

■ 原因A: ASICの学習機能がLinuxを素通りしてしまう(FDB問題)

☒ 問題のイメージ

宅配センター (ASIC) が一度「この荷物はこの棚に届ける」と覚えると、次からは仕分け係 (KNET) を通さずに直接SONiCの制御プログラムに渡してしまう。本来届けたいDockerコンテナには届かない。

技術的な説明

ASICのFDB (転送データベース) にMACアドレスが登録されると、フレームはKNET per-port フィルタをバイパスしてSONiCのorchagentに届く。orchagentはIPルーティング管理プロセスで、Dockerブリッジとは無関係。

解決策A

各物理ポートを1ポートのみの専用VLAN (仮想LAN) に隔離する。同じVLANに他のポートがなければASICは毎回フラッド (全送り) し、FDB学習が起きない。

■ 原因B (主要原因): ASICがVLANタグを付けてLinuxに渡す

☒☒ 問題のイメージ

荷物 (パケット) を届けるとき、配達員 (ASIC) が勝手に「VLAN30番の荷物」という付箋 (VLANタグ) を貼って渡してくる。受け取り口 (eatables) には「付箋が貼ってある荷物は受け取り拒否」というルールがあるため捨てられてしまう。

```
# tcpdumpで確認した実際の出力
# 本来は ethertype ARP のはずが...
ethertype 802.1Q (0x8100), <- VLANタグが付いている!
vlan 30, ethertype ARP,
Reply 10.10.1.1 is-at 3e:03:dd...
```

```
# eatablesのルール (SONiCデフォルト)
rule 16: -p 802_1Q -j DROP <- タグ付きは全部捨てる!
```

なぜタグが付くのか

Broadcom Trident3 ASICの仕様で、cpuポートを「タグなし受信モード (untagged member)」に設定できない。bcmcmdで何度設定コマンドを実行してもvlan showで確認すると反映されていなかった。

■ 原因C: 誤った対処法を試みて別の問題が発生

☒ VLANサブインタフェース方式(失敗)

Ethernet0.30という仮想インタフェースをブリッジに入れ、受信時にタグを自動で外す方法を試みた。受信方向は動いたが、送信方向でKNETがタグ付きパケットをASICに転送できないという新たな問題が発生。tcpdumpで「対向ポートで0パケット」と確認され、送信が完全に機能しなかった。

5 最終的な解決策:tc ingress VLAN pop

☒ 解決のアイデア

付箋(VLANタグ)を「受け取り口(ebtables)の前」で剥がしてしまえばよい。Linuxのtc ingress機能はあらゆる処理より先に動くため、ここでタグを外してからブリッジに渡すと、ebtablesは普通のARPパケットとして通してくれる。

■ 受信・送信それぞれの動き

受信方向 (物理回線→コンテナ)	物理ポートにパケット到着 → ASICがVLANタグを付けてLinuxへ → tc ingressがタグをPOP(剥がす) → ブリッジにタグなしARPとして届く → ebtables ACCEPT → コンテナへ正常に届く ✓
送信方向 (コンテナ→物理回線)	コンテナ → ブリッジ → EthernetXへタグなしで送信 → KNETがタグなしフレームをASICへ転送 → ASICが物理ポートからタグなしで送出 ✓

```
# 各物理ポートに設定するコマンド
tc qdisc add dev EthernetX handle ffff: ingress
tc filter add dev EthernetX parent ffff: \
protocol 802.1Q flower vlan_id <VLAN番号> \
action vlan pop

# SW1: Ethernet0-11 -> VLAN 30-41
# Ethernet12-14 -> VLAN 42-44
# SW2: Ethernet0-2 -> VLAN 42-44 / Ethernet3-8 -> VLAN 20-25
```

6 試行錯誤の記録(タイムライン)

Step 1:静的FDBエントリの手動追加(失敗)

MACアドレスをASICのFDBテーブルに手動登録 →
FDB登録がむしろ問題を悪化させると判明(orchagentへのバイパスを引き起こす)

Step 2:per-port VLAN分離の導入(部分成功)

各ポートを独立VLAN(20-44)に隔離 → FDB問題は解消。しかしVLANタグによるebtables DROPが露見し、根本的な疎通は取れなかった

Step 3:bcmcmdでubm=cpuを設定(失敗)

ASICのVLAN設定でcpuをuntaggedメンバーにしようとした。コマンドはエラーなく通るが、vlan showで確認するとcpuがubmに入っていない。Broadcom Trident3のハードウェア仕様上、不可能な設定だった。

Step 4: VLANサブインタフェース方式(失敗)

Ethernet0.30をブリッジに参加させて受信時にVLANタグを自動除去しようとした。受信側ではフレームが届くことを確認したが、KNETがタグ付きフレームをASICに転送しないため送信方向が完全に機能しなかった。

Step 5: tc ingress flower vlan pop(成功)

物理EthernetXのtc ingress(最前段)にVLANタグを剥がすフィルタを設定。受信・送信ともに正常動作。全パスで0% packet lossを達成!

7 最終結果

すべての接続パスで疎通に成功しました(0% packet loss)。

- ✓ SW1内部:Tx1/Tx2/Tx3 ⇄ LER_Ingress (往復遅延 約0.4ms)
- ✓ SW1内部:LER_Ingress ⇄ CR1/CR2/CR3 (往復遅延 約40ms)
- ✓ SW2内部:LER_Egress ⇄ Rx1/Rx2/Rx3 (往復遅延 約0.3ms)
- ✓ スイッチ間:CR1/CR2/CR3(SW1) ⇄ LER_Egress(SW2) (約0.5ms)
- ✓ エンドツーエンド:Tx1→Rx1, Tx2→Rx2, Tx3→Rx3(MPLSパス) TTL=61、約41ms

```
# 最終確認の出力
$ docker exec Tx1 ping -c3 10.20.1.2
64 bytes from 10.20.1.2: icmp_seq=1 ttl=61 time=40.8 ms
64 bytes from 10.20.1.2: icmp_seq=2 ttl=61 time=41.0 ms
64 bytes from 10.20.1.2: icmp_seq=3 ttl=61 time=40.9 ms
3 packets transmitted, 3 received, 0% packet loss
```

8 技術的まとめ:わかったこと

制約① ASICのcpuはタグなしにできない	Broadcom Trident3 ASICは「cpuポートへのフレームを常にVLANタグ付きで渡す」というハードウェア仕様を持つ。ソフトウェアでは回避できない。
制約② SONiCがVLANタグを捨てる	SONiCのebtablesはデフォルトで802.1Qタグ付きフレームをDROPする。このルールはSONiCの正常動作に必要なため単純には削除できない。
制約③ KNET TXはVLANタグを扱えない	LinuxからASICへの送信パスでは、VLANタグ付きフレームが正しく転送されない。VLANサブインタフェース方式が失敗した原因。
解決策:tc ingress vlan pop	tc ingressフック(最前段)でVLANタグを剥がすことで、ebtablesに到達する前にタグを除去できる。TX方向は元々タグなしなので影響なし。

9 そこから学べること

☒ コマンドが成功＝設定が反映された、とは限らない

bcmcmdでubm=cpuを設定してもエラーは出なかったが、実際には反映されていなかった。設定後は必ずvlan showなどで実際の状態を確認することが重要。

☒ 問題は層（レイヤー）ごとに切り分ける

今回はASIC層・KNET層・eatables層・ブリッジ層が複雑に絡み合っていた。tcpdumpでどの層でパケットが消えるかを特定し、一段ずつ解決した。

☒ 失敗した解決策の「なぜ失敗したか」を理解してから次へ進む

VLANサブインタフェース方式が失敗したとき、原因（KNETのTX制約）をtcpdumpで確認してから次の手（tc ingress）を選んだ。原因不明のまま別の方法を試すと迷走する。

☒ ハードウェアの仕様書がなければ動作確認で仕様を把握する

Broadcom ASICの詳細仕様は非公開。実際にコマンドを打ちvlan showで確認することで「cpuはubmに入れられない」という仕様を発見した。

☒☒ OSの安全機能が実験の邪魔をすることがある

SONiCのeatablesルールは必要なセキュリティ機能だが今回の実験環境では邪魔になった。削除するのではなく、tc ingressで問題を回避するアプローチを選んだ。

実証① パケット優先制御 (QoS) の検証結果

☒ イメージ: 救急車レーン

道路(ネットワーク)が混んでいても、救急車(高優先パケット)は専用レーンで先に通してもらえる。一般車(低優先)は後回しになるが、救急車は確実に目的地(受信コンテナ)に届く。

■ 実験条件

仕組み① DSCPマーキング	LER_Ingressのiptablesが宛先ポート番号を見て優先度ラベルを書き込む。ポート1000 → 高優先 (AF41) / ポート2000 → 中優先 (AF42) / ポート3000 → 低優先 (AF43)
仕組み② WRRキューイング (HTB)	各優先度のパケットを別々のキューに入れ、重み 高4:中2:低1 の比率で帯域を配分。LER_Ingressのleri-crX出力インタフェースとCR1/2/3のcr-lere出力インタフェースに適用。
負荷条件	Tx1/Tx2/Tx3がそれぞれ50Mbpsで10秒間UDPを同時送信(合計150Mbps)。回線の実効帯域は約100Mbps → 意図的に輻輳を発生させてQoSの効果を確認。

■ iperf3測定結果

フロー	優先度	送信	受信	パケットロス
Tx1→Rx1 (port 1000)	AF41 高	50.0 Mbps	49.8 Mbps	0.04%
Tx2→Rx2 (port 2000)	AF42 中	50.0 Mbps	33.4 Mbps	33%
Tx3→Rx3 (port 3000)	AF43 低	50.0 Mbps	7.51 Mbps	85%

実際のiperf3出力(受信側サマリ)

```
[Tx1→Rx1] 0.00-10.04 sec 59.6 MBytes 49.8 Mbits/sec 17/43164 (0.039%) receiver
[Tx2→Rx2] 0.00-10.04 sec 40.0 MBytes 33.4 Mbits/sec 14208/43167 (33%) receiver
[Tx3→Rx3] 0.00-10.29 sec 9.21 MBytes 7.51 Mbits/sec 36433/43101 (85%) receiver
```

☒ 結果の解釈

3フロー合計150Mbpsが100Mbpsの回線に押し込まれた状況で、高優先 (AF41) がほぼ全帯域 (49.8Mbps) を確保。低優先 (AF43) は7.5Mbpsまで絞られた。受信帯域の合計: $49.8 + 33.4 + 7.5 = 90.7$ Mbps (HTBのburst設定による若干の誤差あり)。WRR重み4:2:1が実測値に反映されており、「救急車レーン」が正しく機能することを実証。

実証② 障害時の自動迂回 (Fast Reroute) の検証結果

☒☒ イメージ: カーナビの自動迂回

カーナビ (OSPF) が道路の通行止め (リンク障害) を素早く検知して自動で別ルート案内する。CR1・CR2・CR3の3本の経路があるので、1本が故障しても残り2本に自動切り替えできる。BFD (生死確認プロトコル) が数十ミリ秒ごとに隣のルータに「生きてる?」と確認し、返事がなければ即座に障害と判断してOSPFに通知する。

■ 実装した構成

FRR companionコンテナ	frrouting/frr DockerイメージをLER_Ingress・CR1/2/3・LER_Egressとネットワーク名前空間を共有 (--network container:X) して起動。既存コンテナを改変せずにルーティングデーモンを追加できる構成。
OSPF設定	全ルータをエリア0 (バックボーン) に参加。OSPF hello=1秒、dead=3秒。LER_Ingress→CR1/2/3→LER_EgressのトポロジーをOSPFが自動学習。
BFD設定	OSPFネイバーリンク全てにBFDセッションを設定 (OSPF+BFD連動)。タイマー: 300ms×multiplier 3 = 900ms (OSPFのdead 3秒より高速)。
MPLSバックアップルート	FRR staticdにMPLSラベルルートを登録 (label 100 via leri-cr1)。インタフェース復旧時にstaticdが自動再インストールする (proto 196)。

■ 正常時の状態確認

```
OSPF neighbors (LER_Ingress):
10.0.1.2 Full leri-cr1 (CR1) Up 8m
10.0.3.2 Full leri-cr2 (CR2) Up 14m
10.0.5.2 Full leri-cr3 (CR3) Up 14m

BFD peers: 3 sessions, Status: up (全て正常)

traceroute Tx1 → Rx1 (正常時):
1 10.10.1.1 LER_Ingress 0.2ms
2 10.0.3.2 CR2 40.2ms ← OSPF ECMP でCR2を選択
3 10.0.6.2 LER_Egress 40.4ms
4 10.20.1.2 Rx1 40.9ms
```

■ CR1障害シミュレーション → 自動フェイルオーバー

```
# leri-cr1インタフェースをダウン (CR1障害をシミュレート)
docker exec LER_Ingress ip link set leri-cr1 down

# 直後のルートテーブル (leri-cr1ルートが消え、CR2が最優先に)
10.20.1.0/30 nhid 110 proto ospf ← nexthop IDが62→110に変化
10.20.1.0/24 via 10.0.3.2 leri-cr2 metric 2 ← CR2が最優先
10.20.1.0/24 via 10.0.5.2 leri-cr3 metric 3

# pingの結果 (フェイルオーバー中)
5 packets transmitted, 5 received, 0% packet loss
```

✓ CR1リンクダウン直後: OSPFのnexthop IDが即座に更新 (62→110)

- ✓ leri-cr2経由 (metric 2) に自動切り替え: 0%パケットロスを達成
- ✓ tracerouteの中継ホストが変化: 障害検知→経路切り替えを可視化
- ✓ FRR staticdがインタフェース復旧を検知し、MPLSラベルルートを自動再インストール (proto 196)

☒ Fast Reroute結果まとめ

CR1リンク障害時: Zebraが即座にECMPグループを更新し、CR2経由に自動切り替え。5回のpingで0%パケットロスを達成 (フェイルオーバーが瞬時すぎてpingが捉えられないほど)。インタフェース復旧後: FRR staticdがMPLSラベルルート (label 100) を自動再インストール。人間が何もしなくても、障害→切り替え→回復が自動で行われることを実証。

このレポートのまとめ

① MPLS実験環境構築: Broadcom ASICのVLANタグ問題をtc ingress vlan popで回避。全パスで0%パケットロスを達成 (TTL=61、約41ms)。② QoS実証: 3フロー同時150Mbpsの輻輳環境で、AF41が49.8Mbps (0.04%ロス)、AF43が7.51Mbps (85%ロス)。WRR重み4:2:1が機能することを実測で確認。③ Fast Reroute実証: OSPF+BFD+FRRデーモンにより、CR1障害時に0%パケットロスで自動フェイルオーバー成功。FRR staticdがMPLSルートを自動復元。